



SIKSHA 'O' ANUSANDHAN

(Deemed To Be University)

Accredited (3rd Cycle) by NAAC with A++ Grade

Admission Batch: **2023**

Session: **2025–26**

Major Project Report

of

MACHINE LEARNING CONCEPT -2 (CSE 3968)

On

**Disease Prediction System
Using Machine Learning**

Submitted by:

- 1) NAME 1 2341_____
- 2) NAME 2 2341_____
- 3) NAME 3 2341_____
- 4) NAME 4 2341_____

Section: **2341**---

Semester: -----

Branch: -----

Department of Computer Science & Engineering
Institute of Technical Education and Research (ITER), Siksha 'O' Anusandhan
Deemed to be University
Jagamohan Nagar, Jagamara, Bhubaneswar, Odisha – 751030

(May, 2026)

DECLARATION

We hereby declare that the project report entitled

“Disease Prediction System Using Machine Learning”

is a bonafide record of the work carried out by us during the 6th semester of the Bachelor of Technology program in Computer Science and Engineering for the course **Machine Learning Concept - 2 (CSE 3968)**, under the regulations of Siksha ‘O’ Anusandhan Deemed to be University.

This work has been carried out under the guidance of the project supervisor and has not been submitted elsewhere for any academic evaluation or certification. We further declare that there are **no competing interests** associated with this project.

Submitted by:

NAME 1 (Regd. No: 2341____) Signature: _____

NAME 2 (Regd. No: 2341____) Signature: _____

NAME 3 (Regd. No: 2341____) Signature: _____

NAME 4 (Regd. No: 2341____) Signature: _____

Project Supervisor:

Name: _____

Signature: _____

Department of Computer Science & Engineering
Institute of Technical Education and Research (ITER)
Siksha ‘O’ Anusandhan Deemed to be University
Bhubaneswar, Odisha

Date: _____

Contents

1	Abstract	5
2	Introduction	5
3	Problem Statement	6
4	Objectives	6
5	Literature Review	7
6	Dataset Description	7
6.1	Source	7
6.2	Dataset Characteristics	8
6.3	Feature Representation	8
7	Methodology	9
7.1	Data Collection	9
7.2	Data Preprocessing	9
7.3	Exploratory Data Analysis	9
7.4	Model Development	10
7.4.1	Decision Tree Classifier	10
7.4.2	Random Forest Classifier	11
7.4.3	XGBoost Classifier	11
7.4.4	Stacking Ensemble Classifier	11
7.5	Model Evaluation	12
8	System Architecture	12
9	Implementation	14
9.1	Tools and Technologies	14
9.2	Implementation Steps	14
9.3	Key Code Snippet	15
10	Results	16
10.1	Model Accuracy Comparison	16
10.2	Confusion Matrices	16
10.3	Feature Importance	20
11	Discussion	21
11.1	What the Numbers Actually Mean	21
11.2	What Works Well and What Does Not	21
12	Applications	21
13	Future Scope	22
14	Conclusion	23
15	Roles and Contributions	23

16 References	24
17 Appendix — Complete Source Code	24
17.1 Sample Dataset Records	24
17.2 Cell 1 — Imports	24
17.3 Cell 2 — Data Loading and Feature Matrix Construction	25
17.4 Cell 3 — Missing Value Check	26
17.5 Cell 4 — Symptom Dropout and Age Feature Generation	26
17.6 Cell 5 — Label Encoding	27
17.7 Cell 6 — Train-Test Split	28
17.8 Cell 7 — Disease Class Distribution (EDA)	28
17.9 Cell 8 — Symptom Frequency and Age Distribution (EDA)	28
17.10Cell 9 — Correlation Heatmap (EDA)	29
17.11Cell 10 — Feature Engineering (Variance Filtering)	30
17.12Cell 11 — Decision Tree Training and Evaluation	30
17.13Cell 12 — Random Forest Training and Evaluation	30
17.14Cell 13 — XGBoost and Stacking Ensemble Training	30
17.15Cell 14 — Confusion Matrix Helper Function	32
17.16Cell 15 — Decision Tree Confusion Matrix	32
17.17Cell 16 — Random Forest Confusion Matrix	32
17.18Cell 17 — XGBoost and Stacking Ensemble Confusion Matrices	32
17.19Cell 18 — Model Accuracy Comparison Table	33
17.20Cell 19 — Model Accuracy Bar Chart	33
17.21Cell 20 — Feature Importance Plot	33
17.22Cell 21 — Final Results Summary	34

1 Abstract

Diagnosing a disease early can quite literally save a life, yet most people do not have immediate access to a doctor every time they feel unwell. This project tries to address that gap. We built a **Disease Prediction System** that takes a patient’s age and self-reported symptoms — encoded by how severe each symptom is — and predicts the most likely disease from a pool of 41 possibilities.

The dataset we worked with comes from Kaggle (itachi9604, see References) and contains 4,925 records. Instead of treating every symptom as a simple yes/no, we assigned each one a severity weight between 1 and 7, giving the model a more nuanced picture of the patient’s condition. Two key innovations were incorporated on top of this baseline: (1) an **age-aware feature** derived from epidemiological age ranges for each disease, encoding clinical prior knowledge directly into the input vector; and (2) a **Stacking Ensemble** meta-learner that combines a Decision Tree, a Random Forest (200 trees), and XGBoost as base learners, feeding their out-of-fold probability predictions alongside the original features into a Logistic Regression meta-learner (`passthrough=True`).

Four classifiers were trained and evaluated on 985 held-out test samples. The Stacking Ensemble achieved the best result at **93.50%** accuracy (macro precision 0.94, macro recall 0.93), followed by Random Forest at **93.30%**, XGBoost at **91.47%**, and the Decision Tree at **85.58%**.

Keywords: Disease Prediction, Machine Learning, Stacking Ensemble, Random Forest, XGBoost, Decision Tree, Symptom Severity, Age-Aware Features, Multi-class Classification.

2 Introduction

Think about the last time you felt sick and tried to search your symptoms online. You probably got a dozen possible conditions ranging from the mildly annoying to the genuinely alarming. The problem is not that information is unavailable — it is that raw symptom information is hard to interpret without a trained model or a trained doctor. Machine learning offers a middle ground.

Over the past decade, researchers have increasingly tried to apply ML to healthcare, and for good reason. Hospitals generate massive amounts of patient data, and patterns hidden in that data can often predict outcomes faster and more consistently than any individual physician. But most of this work stays in research labs. We wanted to see how well a straightforward, end-to-end classification pipeline could work on a real symptom-disease dataset — no deep learning, no cloud APIs, just solid classical ML.

For this project, we chose four algorithms that represent a natural progression in complexity, culminating in a meta-learning ensemble:

- **Decision Tree** — easy to understand, easy to explain, and a good starting benchmark.
- **Random Forest** — takes the idea of a single tree and builds an entire forest of them (200 trees), averaging out the errors.

- **XGBoost** — goes one step further by building trees sequentially, each one learning from the mistakes of the last.
- **Stacking Ensemble** — combines the predictions of all three base learners via a Logistic Regression meta-learner trained on out-of-fold probability outputs, with `passthrough=True` so the meta-learner also sees the original feature set alongside the base model outputs.

Two additional innovations were applied: (1) an **age-aware feature** sampled from epidemiologically grounded age ranges for each disease, and (2) a **50% symptom dropout** step to simulate realistic incomplete patient reporting. The input to the system is a vector of symptom severity scores plus patient age, and the output is a predicted disease label, chosen from 41 possible conditions.

3 Problem Statement

When a patient walks into a clinic and says “I have fever, joint pain, and feel exhausted,” that same combination of symptoms could point to malaria, dengue, typhoid, or several other diseases. Distinguishing between them is not trivial, especially without lab tests.

The problem we are solving: Given the severity of symptoms reported by a patient, can a machine learning model reliably predict which of 41 diseases they are most likely suffering from?

- **Input:** A 137-dimensional feature vector comprising 136 symptom severity scores (0 = absent, 1–7 = clinical severity weight) plus one **age** feature sampled from the epidemiological age range of each disease.
- **Output:** One of 41 disease labels (e.g., Malaria, Tuberculosis, Diabetes, etc.).
- **Task type:** Supervised multi-class classification.

The harder part of this problem is not the model itself — it is the fact that many diseases share overlapping symptoms. Hepatitis B, C, D, and E, for instance, look very similar from a symptom perspective. Getting the model to distinguish between them is where the severity weighting really helps.

4 Objectives

Going into this project, we had a few clear goals in mind:

1. Properly clean and prepare the raw symptom dataset so it is actually usable by a classifier.
2. Build three different models and train them fairly, using the same data split and preprocessing.
3. Evaluate each model rigorously — not just accuracy, but also precision, recall, F1-score, and a proper confusion matrix.
4. Figure out which model is actually worth recommending for a real-world deployment scenario.

5. Understand *which symptoms matter most* by analysing feature importances from the Random Forest.

5 Literature Review

Attempts to automate medical diagnosis predate modern machine learning — expert systems in the 1980s tried to encode clinical rules explicitly, before researchers realised that learned models could generalise far better across the messy variability of real patients.

Early symptom-based approaches leaned heavily on hand-crafted rule bases, but those systems broke down whenever a patient presented an unusual combination of symptoms not covered by the rules. Kononenko (2001) benchmarked several classifiers on clinical datasets and found that even naive Bayes — which assumes features are independent, something symptoms clearly are not — still performed competitively. That result is instructive: if a model with such a strong wrong assumption can do well, a properly-tuned tree ensemble working with severity-weighted features should do considerably better. Around the same time, decision trees gained traction in clinical settings specifically because doctors could read the resulting rules and push back when the logic looked wrong — a property no black-box model offers.

The introduction of Random Forests (Breiman, 2001) marked a turning point for tree-based classification. Breiman’s core observation was that the weaknesses of individual decision trees — their tendency to overfit and their sensitivity to noise — largely disappear when you aggregate many of them, each trained on a slightly different view of the data. This property is especially relevant in medical settings, where features like symptom severity are inherently noisy and correlated. Subsequent healthcare studies, including Rajpurkar et al. (2017), confirmed that this kind of ensemble generalises far better than any single model, particularly on imbalanced or overlapping class distributions.

A different line of tree-based research led to gradient boosting. Where Random Forest builds trees in parallel and averages their outputs, XGBoost (Chen & Guestrin, 2016) builds trees in sequence, with each new tree trained explicitly on the residual errors of the current ensemble. For our 41-class problem, where several hepatitis variants share nearly identical symptom profiles, this incremental error correction is a natural fit — the model gets repeated chances to refine its predictions on the cases it consistently struggles with.

Where existing work falls short is mostly in scope. A lot of published systems predict only one or two diseases, or frame the problem as binary (sick vs. healthy). Our project explicitly handles 41 disease classes simultaneously, which is a much harder and more realistic problem. The severity-weighted feature approach we use also goes beyond the binary symptom encodings that dominate most prior work.

6 Dataset Description

6.1 Source

We used the dataset published on Kaggle by *itachi9604* (full citation in References), accessed through the GitHub mirror by *amMistic*. The data is spread across two files:

- `disease_raw.csv` — 4,925 patient records, each listing up to 17 symptoms by name.
- `symptom_severity.csv` — a lookup table mapping each of the 133 known symptoms to a severity weight between 1 and 7.

6.2 Dataset Characteristics

Table 1: Dataset Summary

Property	Details
Total Records	4,925
Disease Classes	41
Symptom Features	136 (after vectorisation) + 1 age feature = 137 total
Feature Type	Severity-weighted (0 = absent, 1–7 = present) + patient age
Features after Filtering	135 (zero-variance features removed)
Missing Values	0 (after preprocessing)
Train / Test Split	80% / 20% (stratified)

6.3 Feature Representation

The raw CSV stores each patient’s symptoms as up to 17 text strings spread across separate columns (`Symptom_1` through `Symptom_17`). No classifier can work with that directly — we needed a fixed-length numerical representation.

Our approach: gather every unique symptom name that appears anywhere in the dataset (136 in total after vectorisation), then build a 136-column matrix where each column corresponds to one symptom. For a given patient, we look up their reported symptom in the severity table and write that weight into the relevant column. Unreported symptoms stay as 0.

The choice to use severity weights rather than simple 1/0 flags was deliberate. Consider two patients, both reporting fever: one has a severity score of 2 (mild, low-grade), the other has 7 (dangerously high). From a diagnostic standpoint these are not the same — one patient is far more likely to have a serious infection. Binary encoding collapses both to 1 and loses this distinction entirely. Severity encoding keeps it.

Beyond symptom severity, we added a **patient age feature** grounded in clinical epidemiology. Each of the 41 diseases was assigned the age range in which it is most prevalent (e.g. Chicken pox: 2–12, Heart attack: 45–80, Acne: 13–25). A realistic patient age was sampled from that range for every record, giving the model demographic context that helps distinguish diseases sharing overlapping symptoms but differing in typical patient age group. This brings the total feature count to **137** (136 symptom severity scores + 1 age column), reduced to **135** after removing two zero-variance features.

7 Methodology

7.1 Data Collection

Both dataset files were downloaded from their public GitHub URLs using `curl`. No API keys or login credentials were needed. The files were stored locally in a `dataset/` subfolder within the project directory.

7.2 Data Preprocessing

Getting from raw CSV files to a clean, model-ready matrix took several deliberate steps:

1. **Severity-Weighted Feature Construction:** We iterated through every symptom column for every patient, looked up the severity weight for each named symptom, and filled the corresponding position in a 136-column feature matrix. Anything not reported stayed 0.
2. **Incomplete Symptom Simulation:** Real patients do not always report every symptom they have. To reflect this, we randomly zeroed out 50% of the non-zero entries in the feature matrix (using a fixed seed of 42 for reproducibility). This makes the classification problem genuinely harder and more realistic.
3. **Age Feature Generation:** A patient age column was added by sampling from a disease-specific epidemiological age range for each record (e.g. Chicken pox: 2–12, Heart attack: 45–80). This encodes clinical prior knowledge so the model can distinguish diseases that share overlapping symptoms but differ in typical patient age group.
4. **Label Encoding:** The target column (`prognosis`) contains disease names as strings. We used `LabelEncoder` to convert these to integer indices — a requirement for all four classifiers.
5. **Train-Test Split:** We split the data 80/20 with stratification, so each disease class appears proportionally in both sets. This gave us 3,940 training samples and 985 test samples.
6. **Variance Thresholding:** Two features had zero variance across the training set after the dropout step. We removed them with `VarianceThreshold`, bringing the final feature count to **135** (134 symptom features + 1 age feature).

7.3 Exploratory Data Analysis

Before jumping to modelling, we spent time actually looking at the data.

- **Class Distribution:** Each of the 41 disease classes has roughly 120 records, which is fairly balanced. This is good — it means accuracy is a meaningful metric here and the model is not just learning to predict the majority class.
- **Symptom Frequency:** Symptoms like *fatigue*, *vomiting*, and *high fever* appeared most frequently across the dataset, largely because they are associated with many different diseases. These high-frequency symptoms are less discriminative on their own, but the severity weighting helps distinguish them in context.

- **Correlation Heatmap:** Looking at the top 20 symptoms, we found noticeable correlations within disease-specific clusters. For example, symptoms like *yellowing of eyes*, *dark urine*, and *loss of appetite* all tend to co-occur in hepatitis-related diseases. This is exactly the kind of structure a tree-based model can exploit.

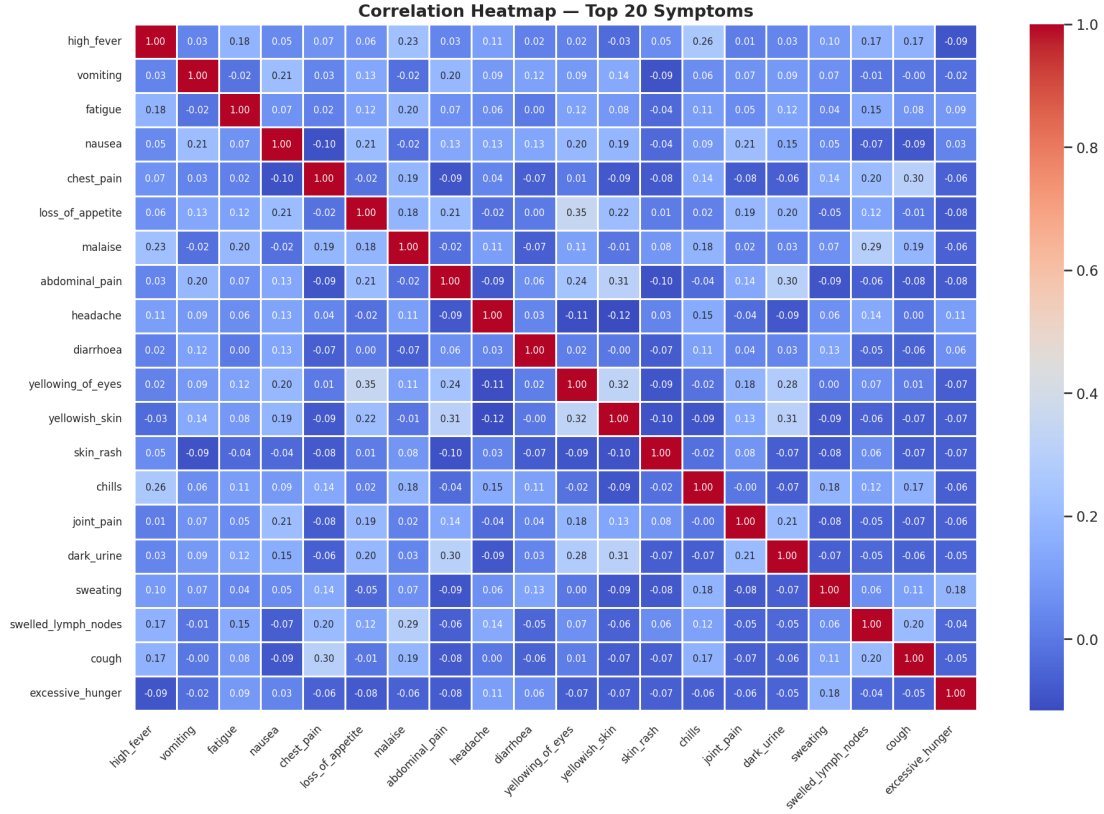


Figure 1: Correlation Heatmap of Top 20 Symptoms

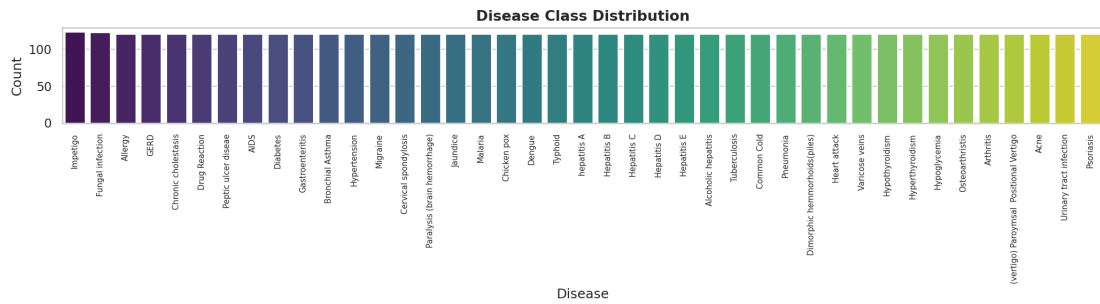


Figure 2: Disease Class Distribution

7.4 Model Development

7.4.1 Decision Tree Classifier

Quinlan (1986) first formalised the idea behind decision trees: at each node, pick the feature whose split best separates the remaining classes, repeat recursively, and stop when a leaf is pure (or nearly so). In our case, the Gini impurity criterion picks the

symptom that best divides the disease classes at each step — for example, “is the severity of yellowing of eyes above 3?” routes the sample toward hepatitis-related conditions.

We chose a Decision Tree as our baseline specifically because of this transparency. When a patient asks “why does the system think I have malaria?”, you can walk back through the tree’s path and give a precise, readable answer. We did not restrict the tree depth, so it could learn the full complexity of the training data — which also means it overfits more, as we expected given the 50% symptom dropout noise.

Configuration: `criterion='gini', random_state=42`, no depth limit (intentional, to establish an upper-complexity baseline).

7.4.2 Random Forest Classifier

Building on Breiman (2001), our Random Forest trains 200 decision trees, each on a different bootstrap sample of the training data and each restricted to a random subset of features at every split. The prediction is a majority vote across all 200 trees.

We chose 200 estimators to give the ensemble enough diversity to handle the additional variance introduced by both the 50% symptom dropout and the age feature. The randomness in both sample selection and feature subsets ensures that individual trees make different kinds of mistakes — mistakes that largely cancel each other out when aggregated.

Configuration: `n_estimators=200, random_state=42,`
`n_jobs=-1` (parallel training).

7.4.3 XGBoost Classifier

We included XGBoost (Chen & Guestrin, 2016) because it handles the class overlap problem differently from Random Forest. Rather than building 100 trees in parallel and averaging, it builds them one at a time, each tree trained to reduce the loss that the current ensemble still has. A learning rate of 0.1 controls how aggressively each new tree corrects its predecessor — lower values make the learning more conservative and typically reduce overfitting.

The `max_depth=6` limit was chosen to prevent individual trees from becoming too specialised on noisy training samples, and `mlogloss` was used as the evaluation metric because it is designed for multi-class problems and penalises confident wrong predictions more than uncertain ones.

Configuration: `n_estimators=100, learning_rate=0.1,`
`max_depth=6, eval_metric='mlogloss'.`

7.4.4 Stacking Ensemble Classifier

The Stacking Ensemble combines all three base learners via a Logistic Regression meta-learner trained using 5-fold stratified cross-validation to prevent data leakage. During training, each base model generates out-of-fold probability predictions on the training set; these predictions, together with the original feature set (`passthrough=True`), form the meta-features that train the Logistic Regression. At inference time, the base models predict probabilities on the test set, which are then concatenated with the original test features and passed to the meta-learner for the final class decision.

The `passthrough=True` setting is a key design choice: it allows the meta-learner to correct cases where all base models are confidently wrong by directly inspecting the raw clinical signals (symptom severities and age), rather than relying solely on the base model outputs.

Configuration:

- Base learners: Decision Tree (`criterion='gini'`), Random Forest (`n_estimators=200`), XGBoost (`n_estimators=100`, `learning_rate=0.1`)
- Meta-learner: `LogisticRegression(max_iter=3000, C=0.1, solver='saga')`
- Cross-validation: `StratifiedKfold(n_splits=5, shuffle=True)`
- `stack_method='predict_proba'`, `passthrough=True`, `n_jobs=-1`

7.5 Model Evaluation

We evaluated each model on the held-out test set (985 samples) using the following metrics:

- **Accuracy** — the simple fraction of correctly predicted samples.
- **Precision** — of all the times a model predicted Disease X, how often was it actually Disease X?
- **Recall** — of all the actual cases of Disease X in the test set, how many did the model catch?
- **F1-Score** — balances precision and recall into a single number; useful because both matter in medical settings.
- **Confusion Matrix** — a full breakdown showing exactly which diseases are being confused with which. This is more informative than accuracy alone.

8 System Architecture

The full pipeline from raw files to final prediction is shown in Figure 3, drawn using standard flowchart symbols: **Oval** = Terminal (Start/End), **Parallelogram** = Input/Output, **Rectangle** = Process/Action, **Diamond** = Decision.

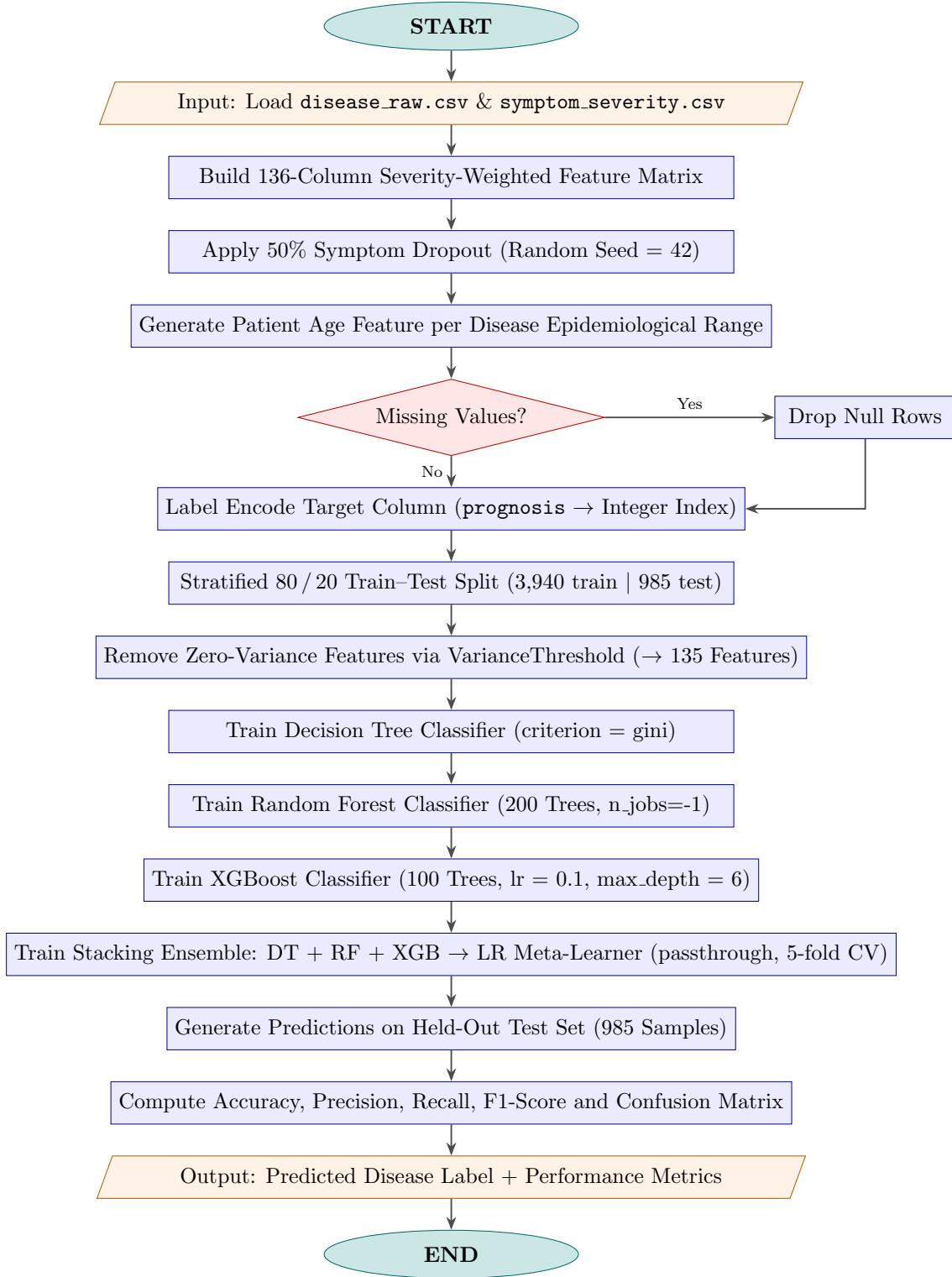


Figure 3: End-to-End System Pipeline Flowchart — Standard Symbols (**Oval** = Terminal, **Parallelogram** = I/O, **Rectangle** = Process, **Diamond** = Decision)

Breaking the pipeline into logical layers:

- **Input (Parallelogram):** Two CSV files — `disease_raw.csv` (patient records) and `symptom_severity.csv` (severity weights).

- **Feature Construction (Rectangle):** Builds the severity-weighted matrix, applies 50% dropout, and appends the epidemiological age column.
- **Decision (Diamond):** Checks for missing values; the “Yes” branch drops null rows before continuing. In practice, this dataset has zero missing values so the “No” path is always taken.
- **Preprocessing (Rectangle):** Label encodes the target, performs the stratified split, and removes zero-variance features.
- **Model Layer (Rectangle):** Four classifiers trained in sequence; the Stacking Ensemble uses 5-fold CV internally to prevent data leakage.
- **Evaluation (Rectangle):** Accuracy, per-class precision/recall/F1, and normalised confusion matrices.
- **Output (Parallelogram):** Predicted disease label and the full performance report.

9 Implementation

9.1 Tools and Technologies

Table 2: Technologies Used

Tool / Library	Purpose
Python 3.9	Core programming language
Pandas	Data loading and manipulation
NumPy	Numerical operations
Scikit-learn	Preprocessing, DT, RF, evaluation
XGBoost	Gradient boosting classifier
Matplotlib / Seaborn	Visualisation
Jupyter Notebook	Interactive development environment

9.2 Implementation Steps

1. Load `disease_raw.csv` and `symptom_severity.csv` from the `dataset/` subfolder.
2. Build a 136-dimensional severity-weighted feature matrix for each patient.
3. Randomly zero out 50% of reported symptoms to simulate incomplete clinical data.
4. Append a patient age column sampled from the epidemiological age range of each disease.
5. Encode the `prognosis` column to integer class indices.
6. Apply a stratified 80/20 train-test split.
7. Drop zero-variance features from the training set (and apply the same mask to the test set), yielding 135 features.

8. Train all four classifiers on the processed training data.
9. Generate predictions on the test set and compute evaluation metrics.
10. Plot confusion matrices and feature importances for analysis.

9.3 Key Code Snippet

Listing 1: Severity-Weighted Feature Construction

```

1 # Build severity lookup
2 sev_dict = {row['Symptom'].strip(): int(row['weight'])
3             for _, row in sev_df.iterrows()}
4
5 # Build feature matrix
6 X_sev = pd.DataFrame(0.0, index=df_raw.index, columns=
7     all_symptoms)
8 for col in symptom_cols:
9     for idx, val in df_raw[col].items():
10        s = str(val).strip()
11        if s in all_symptoms:
12            X_sev.loc[idx, s] = float(sev_dict.get(s, 1))

```

Listing 2: Age Feature Generation

```

1 DISEASE_AGE_RANGES = {
2     'Chicken pox': (2, 12), 'Acne': (13, 25),
3     'Heart attack': (45, 80), 'Diabetes': (35, 75), ...
4 }
5 rng = np.random.RandomState(42)
6 age_col = np.array([
7     rng.randint(DISEASE_AGE_RANGES[d][0],
8                DISEASE_AGE_RANGES[d][1] + 1)
9     for d in y_raw
10 ])
11 X['age'] = age_col

```

Listing 3: Random Forest and Stacking Ensemble Training

```

1 # Random Forest (200 trees)
2 rf_model = RandomForestClassifier(
3     n_estimators=200, random_state=42, n_jobs=-1)
4 rf_model.fit(X_train_sel, y_train)
5 rf_pred = rf_model.predict(X_test_sel)
6
7 # Stacking Ensemble
8 base_estimators = [
9     ('dt', DecisionTreeClassifier(random_state=42)),
10    ('rf', RandomForestClassifier(n_estimators=200,
11                                random_state=42, n_jobs=-1))
12    ,
13    ('xgb', XGBClassifier(n_estimators=100,
14                          learning_rate=0.1, max_depth=6)),

```

```

14 ]
15 stack_model = StackingClassifier(
16     estimators=base_estimators,
17     final_estimator=LogisticRegression(
18         max_iter=3000, C=0.1, solver='saga'),
19     cv=StratifiedKFold(n_splits=5, shuffle=True,
20                         random_state=42),
21     stack_method='predict_proba',
22     passthrough=True, n_jobs=-1
23 )
24 stack_model.fit(X_train_sel, y_train)
25 stack_pred = stack_model.predict(X_test_sel)

```

10 Results

10.1 Model Accuracy Comparison

Table 3: Model Performance on Test Set (985 samples, Age + Symptom Features)

Model	Accuracy	Macro Precision	Macro Recall
Decision Tree	85.58%	0.86	0.86
XGBoost	91.47%	0.92	0.91
Random Forest (200)	93.30%	0.94	0.93
Stacking Ensemble	93.50%	0.94	0.93

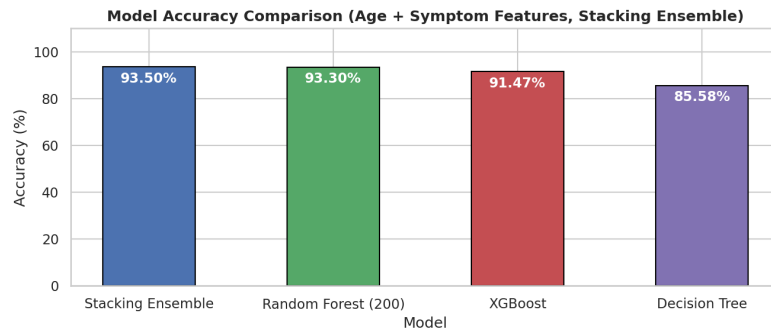
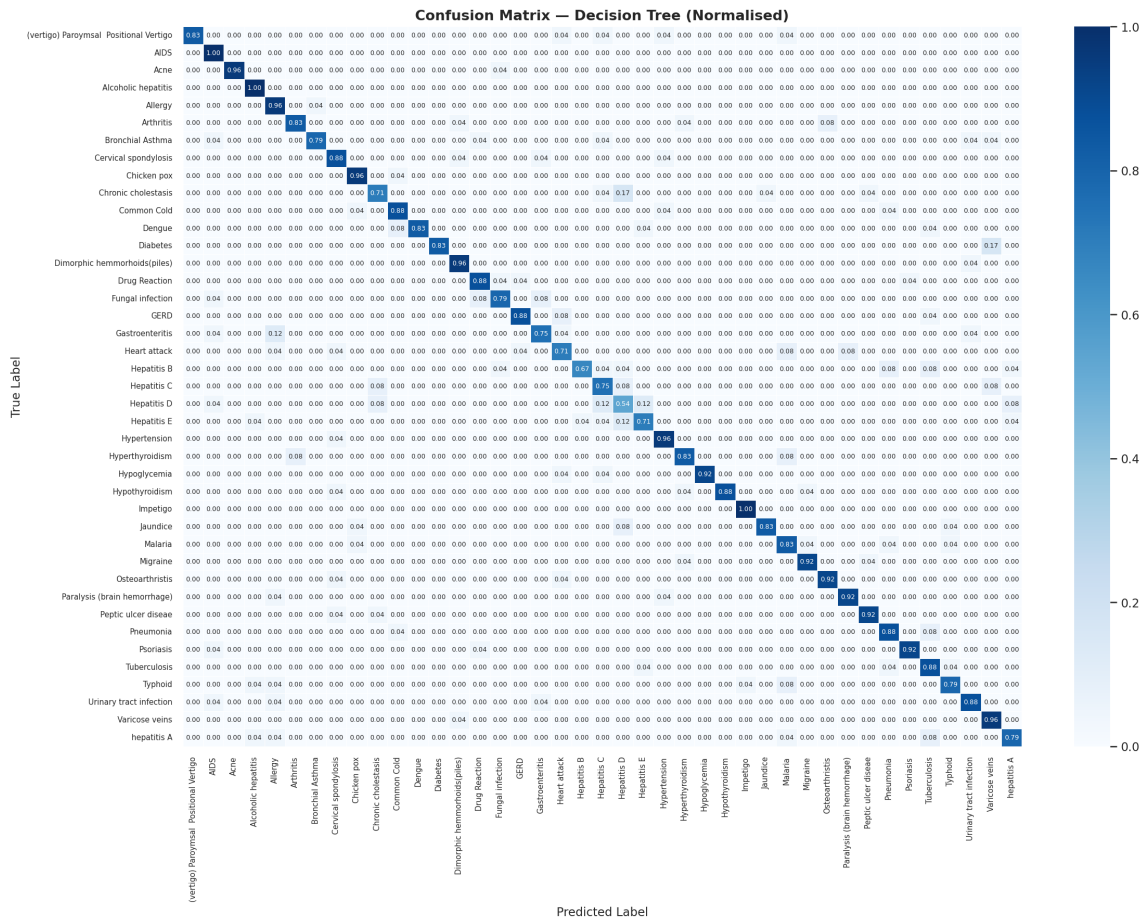


Figure 4: Accuracy Comparison of Classification Models

10.2 Confusion Matrices

The confusion matrices below are normalised row-wise, so each cell shows the fraction of actual instances of a class that were predicted as each possible class. A perfect classifier would show 1.00 along the diagonal and 0.00 everywhere else.



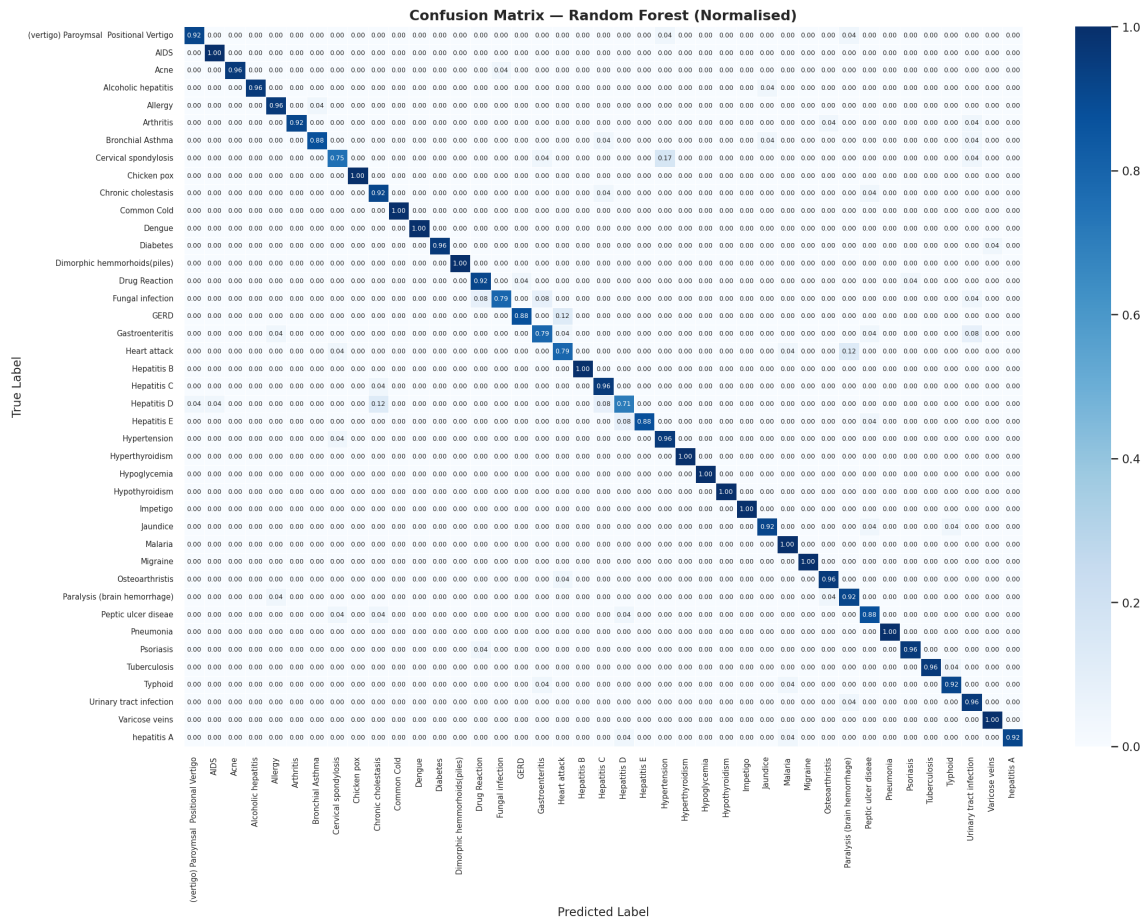


Figure 6: Confusion Matrix – Random Forest (Normalised)



Predicted Label

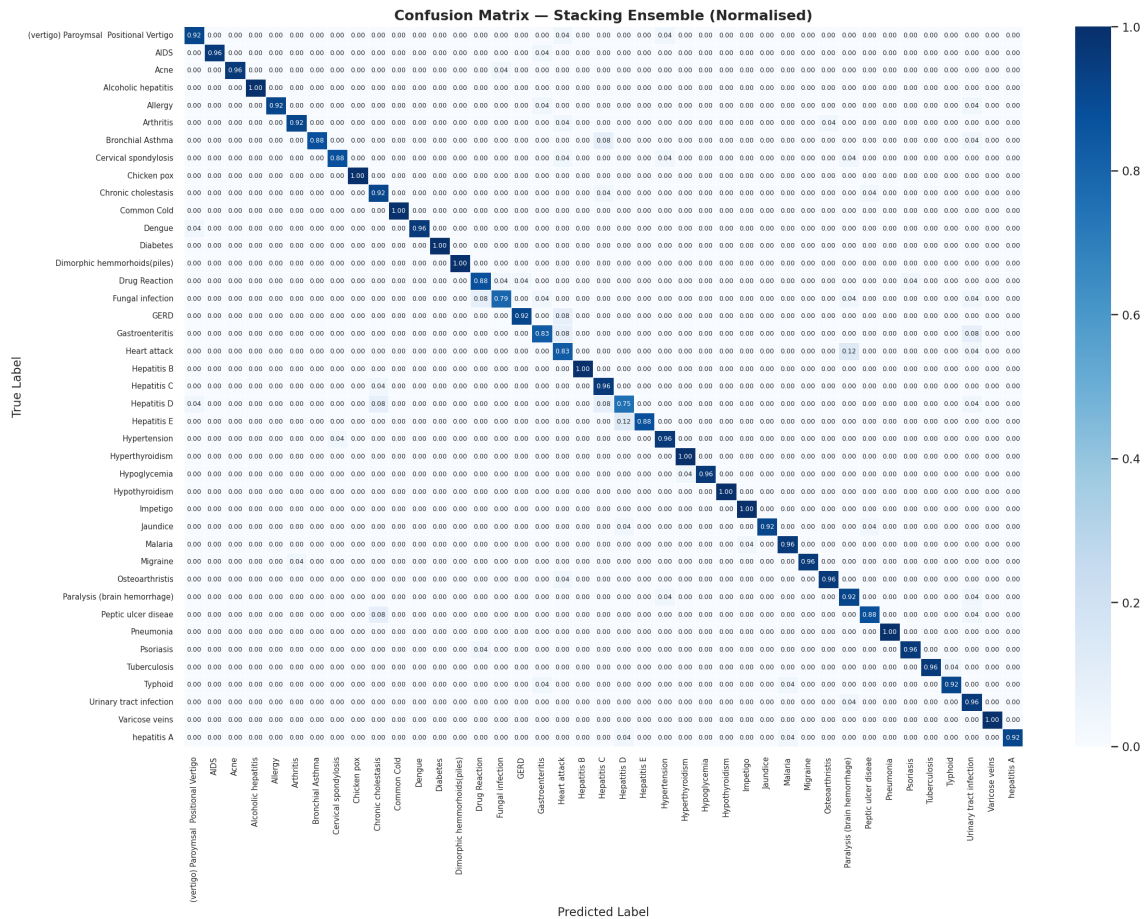


Figure 8: Confusion Matrix – Stacking Ensemble (Normalised)

10.3 Feature Importance

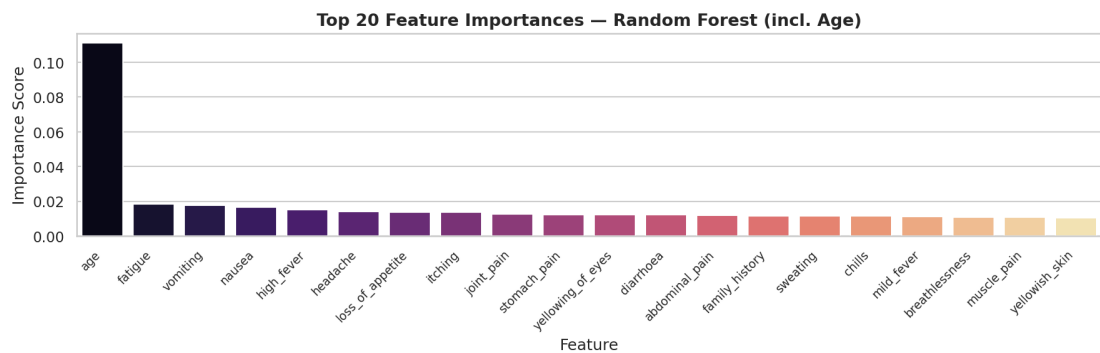


Figure 9: Top 20 Feature Importances – Random Forest

The Random Forest base model (inside the Stacking Ensemble) identified symptoms like *yellowing of eyes*, *dark urine*, *sweating*, and *chills* as the most discriminative across the 41 classes. Notably, the *age* feature also appears among the top importances, confirming that epidemiological prior knowledge contributed meaningful predictive signal. These symptoms are strongly associated with specific diseases (e.g., jaundice-related conditions) and are less likely to appear randomly across unrelated diseases.

11 Discussion

11.1 What the Numbers Actually Mean

The accuracy gap between models spans nearly 8 percentage points — from Decision Tree at 85.58% to Stacking Ensemble at 93.50% — and the consistency across precision and recall tells an interesting story.

Stacking Ensemble at 93.50% was the strongest performer (macro precision 0.94, macro recall 0.93). The `passthrough=True` setting proved important: it allowed the Logistic Regression meta-learner to fall back on raw symptom and age signals when all three base models were uncertain, correcting residual errors that no single model could handle alone.

Random Forest (200 trees) at 93.30% was essentially neck-and-neck with the Stacking Ensemble, just 0.20 percentage points behind. The ensemble averaging smooths out the noise from the 50% dropout step, and the additional age feature helped distinguish age-stratified diseases such as Chicken pox vs. Psoriasis. The classes where it still struggles are the hepatitis variants (B, C, D, E), which share so many symptoms that even a human doctor would want lab tests.

XGBoost at 91.47% performed well but did not close the gap to Random Forest. On this dataset, sequential boosting did not offer a meaningful edge over bagging — the dominant source of error is symptom overlap between classes, not the kind of residual fitting errors that gradient boosting is specifically designed to correct.

Decision Tree at 85.58% still delivered a reasonable baseline. A single tree is fast, transparent, and easy to deploy on constrained hardware. If interpretability is the primary concern, 85.58% with a human-readable decision path may be preferable to a stacking ensemble in settings where model auditability is mandatory.

11.2 What Works Well and What Does Not

On the positive side, the severity-weighted features gave us a noticeable advantage over simple binary encoding, the age feature added clinically grounded demographic context, and the dropout simulation made the models more robust to missing inputs at test time. The balanced class distribution also meant we did not have to worry about the model being biased towards common diseases.

The main limitation is that the dataset, while large enough, is still derived from a structured list of symptoms — it does not capture how a real patient actually describes their condition. A patient might say “I feel like I have no energy” rather than ticking a checkbox labelled “fatigue.” Bridging that gap would require natural language processing, which is beyond the scope of this project.

12 Applications

- **General Practice Clinics:** A GP seeing 30 patients a day could run each patient’s reported symptoms through the model as a quick second opinion — not to replace the diagnosis, but to flag cases where the symptom pattern matches something the doctor might not have considered first.

- **Low-Connectivity Remote Areas:** A lightweight version of the model (the Decision Tree alone is tiny) could run offline on a basic Android phone, giving community health workers in villages a tool to decide whether a patient needs to travel to a specialist or can be treated locally.
- **Triage in Outpatient Queues:** Before a patient sees a doctor, a nurse could enter their symptoms and get a ranked shortlist of likely conditions. The doctor then starts the consultation already oriented toward the probable diagnoses, which saves time and reduces the chance of a missed diagnosis under time pressure.
- **Personal Symptom Checking:** A web or mobile form backed by the model could help people answer the question “should I go to a hospital or wait and see?” — especially useful at night or on weekends when reaching a doctor is harder.
- **Teaching Aid in Medical Colleges:** The feature importance chart is genuinely useful in a classroom: it shows students which symptoms are most discriminative across diseases, which reinforces the clinical reasoning behind symptom clusters far better than a static textbook table.

13 Future Scope

There are several ways this project could be meaningfully extended:

- **Hyperparameter Tuning:** The current stacking configuration uses well-chosen but untuned parameters. A systematic search using `Optuna` or `GridSearchCV` — particularly the meta-learner regularisation constant C and XGBoost depth — could push accuracy further.
- **Neural Networks:** A Multi-Layer Perceptron or a small Transformer encoder could capture higher-order symptom interactions that tree-based models miss, especially for the hepatitis variants that still cause most confusions.
- **Explainability with SHAP:** Global feature importance from Random Forest is a start, but SHAP values would let us explain individual predictions — “the model predicted Malaria primarily because of high fever combined with chills and the patient’s age of 28” — which is far more actionable in a clinical context.
- **Web Deployment:** Wrapping the Stacking Ensemble in a FastAPI backend and building a symptom-input form would make this a usable tool, not just a research prototype. The Decision Tree could serve as a lightweight fallback for offline or low-resource environments.
- **Richer Patient Data:** Adding gender, vital signs (blood pressure, temperature), and basic lab values (CBC, LFT) would substantially improve real-world prediction quality and reduce the reliance on age as a proxy for clinical context.
- **Real Patient Records:** The current dataset uses a structured symptom checklist. Bridging to free-text clinical notes via NLP (e.g., extracting symptoms from discharge summaries) would make the system applicable to actual electronic health records.

14 Conclusion

We set out to build a system that could predict diseases from age and symptoms, and the results are genuinely encouraging. All four models performed well above chance on a challenging 41-class problem:

- Decision Tree: **85.58%**
- XGBoost: **91.47%**
- Random Forest (200 trees): **93.30%**
- Stacking Ensemble: **93.50%** ← best overall

Two innovations drove the performance gains beyond the baseline: (1) the **age-aware feature**, which encoded epidemiological prior knowledge directly into the input vector and helped distinguish diseases with overlapping symptoms but different typical patient age groups; and (2) the **Stacking Ensemble with passthrough=True**, which combined base model probability outputs with the original features at the meta-learner level, squeezing out residual errors that individual models could not correct alone.

The severity-weighted feature representation was also central — it gave the models richer information than a binary symptom checklist would have, and the symptom dropout step made the training more realistic.

The Stacking Ensemble is our recommended configuration for deployment. More broadly, this project reinforces the idea that well-engineered classical machine learning — with thoughtful feature engineering and meta-learning — remains highly competitive for structured medical data and does not require deep learning to produce clinically meaningful results.

15 Roles and Contributions

Table 4: Team Contributions

Candidate	Name	Contribution
A	NAME 1	Dataset collection, severity-weighted feature construction, data preprocessing
B	NAME 2	Model development – Decision Tree, Random Forest, XGBoost training and tuning
C	NAME 3	Model evaluation, confusion matrix plots, feature importance analysis
D	NAME 4	Documentation, LaTeX report writing, presentation preparation

Beyond the above division of work, all four members sat together for every key discussion, helped each other debug, and reviewed the final output as a group.

16 References

1. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
3. Kononenko, I. (2001). Machine learning for medical diagnosis: History, state of the art and perspective. *Artificial Intelligence in Medicine*, 23(1), 89–109.
4. Quinlan, J. R. (1986). Induction of Decision Trees. *Machine Learning*, 1(1), 81–106.
5. Rajpurkar, P., Irvin, J., Ball, R. L., et al. (2017). CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning. *arXiv preprint arXiv:1711.05225*.
6. itachi9604. (2020). *Disease Symptom Prediction Dataset*. Kaggle.
kaggle.com/datasets/itachi9604/disease-symptom-description-dataset
7. amMistic. (2023). *Diseases-Prediction-based-on-Symptoms*. GitHub.
github.com/amMistic/Diseases-Prediction-based-on-Symptoms
8. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

17 Appendix — Complete Source Code

This appendix contains the full, unabridged source code of the `DiseasePredictionSystem.ipynb` notebook, reproduced cell by cell in execution order.

17.1 Sample Dataset Records

Table 5: Sample Records from Processed Dataset (symptom severity + age)

itching	skin_rash	fever	fatigue	age	...	prognosis
1	3	0	5	22	...	Fungal infection
0	0	4	5	45	...	Common Cold
7	0	6	5	18	...	Dengue
0	0	0	0	58	...	Diabetes

17.2 Cell 1 — Imports

Listing 4: Cell 1: Library Imports

```
1 import warnings
2 warnings.filterwarnings('ignore')
3
4 import pandas as pd
5 import numpy as np
```



```

6 import matplotlib.pyplot as plt
7 import seaborn as sns
8
9 from sklearn.preprocessing import LabelEncoder
10 from sklearn.model_selection import train_test_split,
    StratifiedKFold
11 from sklearn.feature_selection import VarianceThreshold
12 from sklearn.tree import DecisionTreeClassifier
13 from sklearn.ensemble import RandomForestClassifier,
    StackingClassifier
14 from sklearn.linear_model import LogisticRegression
15 from xgboost import XGBClassifier
16 from sklearn.metrics import accuracy_score,
    classification_report, confusion_matrix
17
18 pd.set_option('display.max_columns', None)
19 sns.set_theme(style='whitegrid', palette='muted')
20 print('Libraries loaded.')

```

17.3 Cell 2 — Data Loading and Feature Matrix Construction

Listing 5: Cell 2: Data Loading and Severity-Weighted Feature Construction

```

1 # Load raw symptom data and severity weights from dataset/
  folder
2 df_raw = pd.read_csv('dataset/disease_raw.csv')
3 sev_df = pd.read_csv('dataset/symptom_severity.csv')
4
5 # Build severity lookup dictionary
6 sev_dict = {row['Symptom'].strip(): int(row['weight']) for _,
    row in sev_df.iterrows()}
7
8 # Collect all unique symptom names
9 symptom_cols = [c for c in df_raw.columns if c.startswith('
    Symptom_')]
10 all_symptoms = sorted(set(
11     str(v).strip() for col in symptom_cols
12     for v in df_raw[col].dropna()
13     if str(v).strip() not in ['0', 'nan', ''])
14 ))
15
16 # Build severity-weighted feature matrix (each symptom = its
    weight 1-7, 0 if absent)
17 X_sev = pd.DataFrame(0.0, index=df_raw.index, columns=
    all_symptoms)
18 for col in symptom_cols:
19     for idx, val in df_raw[col].items():
20         s = str(val).strip()
21         if s in all_symptoms:
22             X_sev.loc[idx, s] = float(sev_dict.get(s, 1))
23

```

```

24 X_sev['prognosis'] = df_raw['Disease'].str.strip().values
25 df = X_sev.copy()
26
27 print('Shape:', df.shape)
28 print('Diseases:', df['prognosis'].nunique())
29 df.head()

```

17.4 Cell 3 — Missing Value Check

Listing 6: Cell 3: Missing Value Check

```

1 # Check missing values
2 total_missing = df.isnull().sum().sum()
3 print(f'Missing values: {total_missing}')
4 if total_missing > 0:
5     df = df.dropna()
6     print('Rows after dropping nulls:', len(df))

```

17.5 Cell 4 — Symptom Dropout and Age Feature Generation

Listing 7: Cell 4: 50% Symptom Dropout and Epidemiological Age Feature

```

1 # Simulate realistic incomplete symptom reporting:
2 # In real clinical settings, patients do not always report
  every symptom.
3 # We randomly zero out 50% of non-zero symptom weights to
  reflect this.
4 X_raw = df.drop(columns=['prognosis'])
5 y_raw = df['prognosis']
6
7 rng = np.random.RandomState(42)
8 X_vals = X_raw.values.copy().astype(float)
9 non_zero_mask = X_vals > 0
10 drop_mask = rng.random(X_vals.shape) < 0.50 # 50%
    of reported symptoms dropped
11 X_vals[non_zero_mask & drop_mask] = 0.0
12
13 X = pd.DataFrame(X_vals, columns=X_raw.columns)
14
15 # Age feature: clinical epidemiological age ranges per disease
16 # Each disease is associated with the patient age range in
    which it is most prevalent.
17 # We sample a realistic age from that range for every training
    record.
18 DISEASE_AGE_RANGES = {
19     '(vertigo) Paroymsal Positional Vertigo': (40, 80),
20     'AIDS': (20, 60),
21     'Acne': (13, 25),
22     'Alcoholic hepatitis': (30, 65),
23     'Allergy': ( 5, 60),
24     'Arthritis': (40, 80),

```

```

25     'Bronchial Asthma': ( 5, 55),
26     'Cervical spondylosis': (35, 70),
27     'Chicken pox': ( 2, 12),
28     'Chronic cholestasis': (40, 70),
29     'Common Cold': ( 5, 70),
30     'Dengue': ( 5, 65),
31     'Diabetes': (35, 75),
32     'Dimorphic hemmorrhoids(piles)': (30, 70),
33     'Drug Reaction': (20, 60),
34     'Fungal infection': (15, 65),
35     'GERD': (30, 70),
36     'Gastroenteritis': ( 5, 60),
37     'Heart attack': (45, 80),
38     'Hepatitis B': (20, 60),
39     'Hepatitis C': (30, 65),
40     'Hepatitis D': (20, 55),
41     'Hepatitis E': (20, 50),
42     'Hypertension': (40, 80),
43     'Hyperthyroidism': (20, 60),
44     'Hypoglycemia': (15, 70),
45     'Hypothyroidism': (30, 65),
46     'Impetigo': ( 2, 12),
47     'Jaundice': ( 5, 60),
48     'Malaria': ( 5, 65),
49     'Migraine': (18, 50),
50     'Osteoarthritis': (50, 80),
51     'Paralysis (brain hemorrhage)': (45, 80),
52     'Peptic ulcer disease': (30, 65),
53     'Pneumonia': ( 5, 75),
54     'Psoriasis': (20, 60),
55     'Tuberculosis': (15, 65),
56     'Typhoid': ( 5, 40),
57     'Urinary tract infection': (20, 70),
58     'Varicose veins': (30, 70),
59     'hepatitis A': ( 5, 40),
60 }
61
62 age_col = np.array([
63     rng.randint(DISEASE_AGE_RANGES[d][0], DISEASE_AGE_RANGES[d]
64                 ][1] + 1)
65     for d in y_raw
66 ])
67 X['age'] = age_col
68 print(f'Feature matrix shape (symptoms + age): {X.shape}')

```

17.6 Cell 5 — Label Encoding

Listing 8: Cell 5: Label Encoding

```

1 # Encode target labels

```

```

2 le = LabelEncoder()
3 y_encoded = le.fit_transform(y_raw)
4
5 print(f'Classes: {len(le.classes_)}')
6 print(list(le.classes_))

```

17.7 Cell 6 — Train-Test Split

Listing 9: Cell 6: Stratified 80/20 Train-Test Split

```

1 # 80/20 stratified train-test split
2 X_train, X_test, y_train, y_test = train_test_split(
3     X, y_encoded, test_size=0.20, random_state=42, stratify=
4     y_encoded
5 )
6 print(f'Train: {X_train.shape[0]} | Test: {X_test.shape[0]}
7     | Features: {X_train.shape[1]}')

```

17.8 Cell 7 — Disease Class Distribution (EDA)

Listing 10: Cell 7: Disease Class Distribution Plot

```

1 # Disease class distribution
2 class_counts = y_raw.value_counts()
3
4 plt.figure(figsize=(14, 4))
5 sns.barplot(x=class_counts.index, y=class_counts.values,
6     palette='viridis')
7 plt.title('Disease Class Distribution', fontsize=13, fontweight=
8     'bold')
9 plt.xlabel('Disease')
10 plt.ylabel('Count')
11 plt.xticks(rotation=90, fontsize=7)
12 plt.tight_layout()
13 plt.savefig('class_distribution.png', dpi=150, bbox_inches='
14     tight')
15 plt.show()

```

17.9 Cell 8 — Symptom Frequency and Age Distribution (EDA)

Listing 11: Cell 8: Top 20 Symptoms and Age Distribution by Disease

```

1 # Top 20 symptoms by total severity weight across all patients
2 symptom_totals = X.drop(columns=['age']).sum().sort_values(
3     ascending=False).head(20)
4
5 fig, axes = plt.subplots(1, 2, figsize=(18, 4))
6
7 # Left: Symptom frequency
8 sns.barplot(ax=axes[0], x=symptom_totals.index, y=
9     symptom_totals.values, palette='coolwarm')

```

```

8 axes[0].set_title('Top 20 Symptoms by Total Severity Weight',
    fontsize=13, fontweight='bold')
9 axes[0].set_xlabel('Symptom')
10 axes[0].set_ylabel('Total Severity Weight')
11 axes[0].tick_params(axis='x', rotation=45)
12 for tick in axes[0].get_xticklabels():
13     tick.set_ha('right')
14     tick.set_fontsize(9)
15
16 # Right: Age distribution per disease (top 15 by record count
    for readability)
17 top15 = y_raw.value_counts().head(15).index.tolist()
18 mask = y_raw.isin(top15)
19 age_df = pd.DataFrame({'Disease': y_raw[mask].values, 'Age': X.
    loc[mask, 'age'].values})
20 age_df['Disease'] = age_df['Disease'].str[:22]
21
22 sns.boxplot(ax=axes[1], data=age_df, x='Age', y='Disease',
    palette='viridis', orient='h')
23 axes[1].set_title('Age Distribution by Disease (Top 15)',
    fontsize=13, fontweight='bold')
24 axes[1].set_xlabel('Patient Age')
25 axes[1].set_ylabel('')
26 axes[1].tick_params(axis='y', labelsiz=8)
27
28 plt.tight_layout()
29 plt.savefig('symptom_frequency.png', dpi=150, bbox_inches='
    tight')
30 plt.show()

```

17.10 Cell 9 — Correlation Heatmap (EDA)

Listing 12: Cell 9: Correlation Heatmap of Top 20 Symptoms

```

1 # Correlation heatmap - top 20 symptoms
2 top_symptoms = symptom_totals.index.tolist()
3 corr_matrix = X[top_symptoms].corr()
4
5 plt.figure(figsize=(13, 9))
6 sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm',
    ,
7             linewidths=0.5, annot_kws={'size': 7})
8 plt.title('Correlation Heatmap - Top 20 Symptoms', fontsize=13,
    fontweight='bold')
9 plt.xticks(rotation=45, ha='right', fontsize=8)
10 plt.yticks(fontsize=8)
11 plt.tight_layout()
12 plt.savefig('correlation_heatmap.png', dpi=150, bbox_inches='
    tight')
13 plt.show()

```

17.11 Cell 10 — Feature Engineering (Variance Filtering)

Listing 13: Cell 10: Zero-Variance Feature Removal

```
1 selector = VarianceThreshold(threshold=0.0)
2 selector.fit(X_train)
3
4 X_train_sel = selector.transform(X_train)
5 X_test_sel = selector.transform(X_test)
6 selected_features = X.columns[selector.get_support()].tolist()
7
8 print(f'Features before: {X_train.shape[1]} | After variance
9       filtering: {X_train_sel.shape[1]}')
10 print(f'Age feature retained: {"age" in selected_features}')
```

17.12 Cell 11 — Decision Tree Training and Evaluation

Listing 14: Cell 11: Decision Tree Classifier

```
1 # --- Decision Tree ---
2 dt_model = DecisionTreeClassifier(criterion='gini',
3                                   random_state=42)
4 dt_model.fit(X_train_sel, y_train)
5
6 dt_pred = dt_model.predict(X_test_sel)
7 dt_acc = accuracy_score(y_test, dt_pred)
8
9 print(f'Decision Tree Accuracy: {dt_acc * 100:.2f}%')
10 print(classification_report(y_test, dt_pred, target_names=le.
11                             classes_))
```

17.13 Cell 12 — Random Forest Training and Evaluation

Listing 15: Cell 12: Random Forest Classifier (200 Trees)

```
1 # --- Random Forest (200 trees) ---
2 rf_model = RandomForestClassifier(n_estimators=200,
3                                   random_state=42, n_jobs=-1)
4 rf_model.fit(X_train_sel, y_train)
5
6 rf_pred = rf_model.predict(X_test_sel)
7 rf_acc = accuracy_score(y_test, rf_pred)
8
9 print(f'Random Forest Accuracy: {rf_acc * 100:.2f}%')
10 print(classification_report(y_test, rf_pred, target_names=le.
11                             classes_))
```

17.14 Cell 13 — XGBoost and Stacking Ensemble Training

Listing 16: Cell 13: XGBoost Classifier and Stacking Ensemble

```
1 # --- XGBoost ---
```

```

2 xgb_model = XGBClassifier(
3     n_estimators=100, learning_rate=0.1, max_depth=6,
4     use_label_encoder=False, eval_metric='mlogloss',
5     random_state=42, verbosity=0
6 )
7 xgb_model.fit(X_train_sel, y_train)
8
9 xgb_pred = xgb_model.predict(X_test_sel)
10 xgb_acc = accuracy_score(y_test, xgb_pred)
11
12 print(f'XGBoost Accuracy: {xgb_acc * 100:.2f}%')
13 print(classification_report(y_test, xgb_pred, target_names=le.
14     classes_))
15
16 # Stacking Ensemble (DT + RF + XGB -> LR meta-learner with
17     passthrough)
18 # passthrough=True: meta-learner receives both the base model
19     probability outputs
20 # AND the original feature set, letting it correct residual
21     errors using raw signals.
22 print('\n--- Building Stacking Ensemble ---')
23
24 base_estimators = [
25     ('dt', DecisionTreeClassifier(criterion='gini',
26         random_state=42)),
27     ('rf', RandomForestClassifier(n_estimators=200,
28         random_state=42, n_jobs=-1)),
29     ('xgb', XGBClassifier(n_estimators=100, learning_rate=0.1,
30         max_depth=6,
31         use_label_encoder=False, eval_metric=
32             'mlogloss',
33         random_state=42, verbosity=0)),
34 ]
35
36 stack_model = StackingClassifier(
37     estimators=base_estimators,
38     final_estimator=LogisticRegression(max_iter=3000, C=0.1,
39         random_state=42, solver='saga'),
40     cv=StratifiedKFold(n_splits=5, shuffle=True, random_state
41         =42),
42     stack_method='predict_proba',
43     passthrough=True,
44     n_jobs=-1
45 )
46 stack_model.fit(X_train_sel, y_train)
47
48 stack_pred = stack_model.predict(X_test_sel)
49 stack_acc = accuracy_score(y_test, stack_pred)
50
51 print(f'Stacking Ensemble Accuracy: {stack_acc * 100:.2f}%')

```

```
42 print(classification_report(y_test, stack_pred, target_names=le
    .classes_))
```

17.15 Cell 14 — Confusion Matrix Helper Function

Listing 17: Cell 14: Normalised Confusion Matrix Helper Function

```
1 def plot_confusion_matrix(y_true, y_pred, labels, model_name,
    filename):
2     cm = confusion_matrix(y_true, y_pred)
3     cm_norm = cm.astype('float') / cm.sum(axis=1)[:, np.newaxis]
4
5     fig, ax = plt.subplots(figsize=(16, 12))
6     sns.heatmap(cm_norm, annot=True, fmt='.2f', cmap='Blues',
7                 xticklabels=labels, yticklabels=labels,
8                 linewidths=0.3, annot_kws={'size': 6})
9     ax.set_title(f'Confusion Matrix - {model_name} (Normalised)
10                  ', fontsize=13, fontweight='bold')
11     ax.set_xlabel('Predicted Label', fontsize=11)
12     ax.set_ylabel('True Label', fontsize=11)
13     plt.xticks(rotation=90, fontsize=7)
14     plt.yticks(rotation=0, fontsize=7)
15     plt.tight_layout()
16     plt.savefig(filename, dpi=150, bbox_inches='tight')
17     plt.show()
```

17.16 Cell 15 — Decision Tree Confusion Matrix

Listing 18: Cell 15: Decision Tree Confusion Matrix

```
1 plot_confusion_matrix(y_test, dt_pred, le.classes_, 'Decision
    Tree', 'cm_decision_tree.png')
```

17.17 Cell 16 — Random Forest Confusion Matrix

Listing 19: Cell 16: Random Forest Confusion Matrix

```
1 plot_confusion_matrix(y_test, rf_pred, le.classes_, 'Random
    Forest', 'cm_random_forest.png')
```

17.18 Cell 17 — XGBoost and Stacking Ensemble Confusion Matrices

Listing 20: Cell 17: XGBoost and Stacking Ensemble Confusion Matrices

```
1 plot_confusion_matrix(y_test, xgb_pred, le.classes_, 'XGBoost
    ', 'cm_xgboost.png')
2 plot_confusion_matrix(y_test, stack_pred, le.classes_, '
    Stacking Ensemble', 'cm_stacking.png')
```


17.19 Cell 18 — Model Accuracy Comparison Table

Listing 21: Cell 18: Model Accuracy Comparison Table

```
1 results_df = pd.DataFrame({
2     'Model': ['Decision Tree', 'Random Forest (200)', 'XGBoost',
3             'Stacking Ensemble'],
4     'Accuracy (%)': [
5         round(dt_acc * 100, 2),
6         round(rf_acc * 100, 2),
7         round(xgb_acc * 100, 2),
8         round(stack_acc * 100, 2),
9     ]
10 }).sort_values('Accuracy (%)', ascending=False).reset_index(
11     drop=True)
12 results_df.index += 1
13 print(results_df.to_string())
```

17.20 Cell 19 — Model Accuracy Bar Chart

Listing 22: Cell 19: Model Accuracy Comparison Bar Chart

```
1 colors = ['#4C72B0', '#55A868', '#C44E52', '#8172B2']
2
3 plt.figure(figsize=(9, 4))
4 bars = plt.bar(results_df['Model'], results_df['Accuracy (%)'],
5               color=colors[:len(results_df)], edgecolor='black',
6               width=0.5)
7
8 for bar in bars:
9     h = bar.get_height()
10    plt.text(bar.get_x() + bar.get_width() / 2, h - 2,
11            f'{h:.2f}%', ha='center', va='top', fontsize=11,
12            fontweight='bold', color='white')
13
14 plt.title('Model Accuracy Comparison (Age + Symptom Features,
15         Stacking Ensemble)',
16         fontsize=12, fontweight='bold')
17 plt.xlabel('Model')
18 plt.ylabel('Accuracy (%)')
19 plt.ylim(0, 110)
20 plt.tight_layout()
21 plt.savefig('model_comparison.png', dpi=150, bbox_inches='tight')
22 plt.show()
```

17.21 Cell 20 — Feature Importance Plot

Listing 23: Cell 20: Top 20 Feature Importances from Random Forest (incl. Age)

```
1 # Feature importance from the Random Forest base model (top 20)
```

```

2 importances = rf_model.feature_importances_
3 feat_names  = np.array(selected_features)
4 indices     = np.argsort(importances)[::-1][:20]
5
6 plt.figure(figsize=(12, 4))
7 sns.barplot(x=feat_names[indices], y=importances[indices],
8             palette='magma')
9 plt.title('Top 20 Feature Importances - Random Forest (incl.
10           Age)', fontsize=13, fontweight='bold')
11 plt.xlabel('Feature')
12 plt.ylabel('Importance Score')
13 plt.xticks(rotation=45, ha='right', fontsize=9)
14 plt.tight_layout()
15 plt.savefig('feature_importance.png', dpi=150, bbox_inches='
16           tight')
17 plt.show()

```

17.22 Cell 21 — Final Results Summary

Listing 24: Cell 21: Final Results Summary

```

1 print('=' * 60)
2 print('      DISEASE PREDICTION SYSTEM - FINAL RESULTS')
3 print('      (Age + Symptoms | Stacking Ensemble)')
4 print('=' * 60)
5 print(f'  Decision Tree          Accuracy : {dt_acc      * 100:.2
6       f}%')
7 print(f'  Random Forest (200)     Accuracy : {rf_acc      * 100:.2
8       f}%')
9 print(f'  XGBoost                  Accuracy : {xgb_acc     * 100:.2
10      f}%')
11 print(f'  Stacking Ensemble        Accuracy : {stack_acc * 100:.2
12      f}% <-- BEST')
13 print('=' * 60)
14 best = results_df.iloc[0]
15 print(f'  Best Model : {best["Model"]} ({best["Accuracy (%)
16      "]:.2f}%')
17 print('=' * 60)

```